

FLUTTER'DA DURUM YÖNETİMİ

Tema Değiştirme Uygulaması Raporu

Üniversite: Erciyes Üniversitesi

Fakülte: Mühendislik Fakültesi

Bölüm: Bilgisayar mühendisliği

Ders: Mobile Application Development

Öğretim Üyesi: Dr. Öğr. Üyesi Fehim Köylü

Öğrenci: Gökalp Çataldağ

Öğrenci No: 1030521024

Bu doküman, Mobile Application Development dersi kapsamında proje ödevi olarak hazırlanmıştır.

İçindekiler

1. Giriş
2. Durum Yönetimi Nedir?
3. Flutter'da Durum Yönetimi Yaklaşımları
4. ValueNotifier ile Durum Yönetimi
5. Tema Değiştirme Uygulaması
6. Kod Açıklaması
7. Diğer Durum Yönetimi Alternatifler
8. Sonuç
9. Kaynaklar

Giriş

Bu rapor, Flutter uygulamalarında durum yönetiminin önemini ve uygulanmasını incelemektedir. Özellikle, karanlık ve aydınlık tema arasında geçiş yapabilen bir uygulama örneği üzerinden durum yönetimi kavramları açıklanmıştır. Uygulamada ValueNotifier ve ValueListenableBuilder kullanılarak basit ancak etkili bir durum yönetimi mekanizması gösterilmiştir.

Durum Yönetimi Nedir?

Durum yönetimi, bir uygulamanın verilerinin ve kullanıcı arayüzünün nasıl değiştiğini ve bu değişikliklerin nasıl yönetildiğini ifade eder. Flutter uygulamalarında, durum (state) şu şekilde tanımlanabilir:

- **Durum:** Uygulamanın belirli bir anındaki veri konfigürasyonu
- **Durum Yönetimi:** Bu verilerin nasıl depolanacağı, güncelleceği ve UI'a nasıl yansıtılacağı

Etkili durum yönetimi, kodun daha düzenli, bakımı kolay ve ölçeklenebilir olmasını sağlar.

Flutter'da Durum Yönetimi Yaklaşımları

Flutter'da durum yönetimi için birçok yaklaşım mevcuttur:

1. Temel Yaklaşımlar

- **StatefulWidget ve setState():** Flutter'ın dahili durum yönetim sistemi
- **InheritedWidget:** Widget ağacında veri aktarımı için
- **ValueNotifier ve ValueListenableBuilder:** Değer değişikliklerini dinleme ve UI'ı güncelleme

2. Gelişmiş Paketler/Yaklaşımlar

- **Provider:** InheritedWidget üzerine inşa edilmiş, kullanımı kolay bir çözüm
- **Riverpod:** Provider'ın gelişmiş ve daha güvenli versiyonu

- **Bloc/Cubit:** Reactive Programming prensiplerine dayanan, iş mantığını UI'dan ayıran yapı
- **GetX:** State management, route management ve dependency injection sunan kapsamlı çözüm
- **Redux:** Merkezi bir store kullanarak tek yönlü veri akışı sağlayan yaklaşım
- **MobX:** Observable programlama prensipleriyle çalışan reaktif durum yönetimi

ValueNotifier ile Durum Yönetimi

Bu projede kullanılan ValueNotifier, Flutter'ın basit ama güçlü bir durum yönetimi yaklaşımıdır:

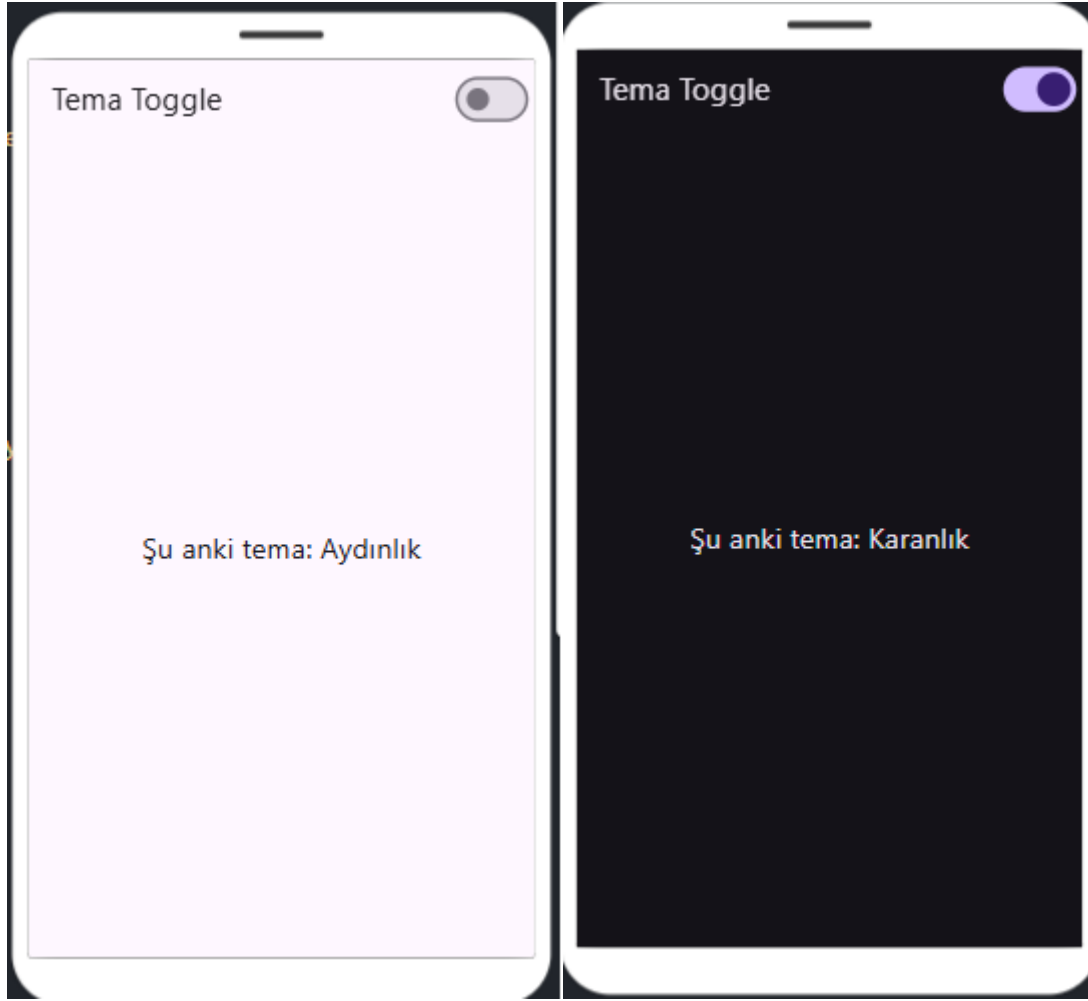
- **ValueNotifier:** Bir değerin değişimini izleyen ve değiştiğinde dinleyicilere bildirim gönderen bir sınıf
- **ValueListenableBuilder:** ValueNotifier'ı dinleyen ve değer değiştiğinde widget ağacını yeniden oluşturan bir widget

Bu yaklaşımın avantajları:

- Basit ve anlaşılması kolay
- Flutter'ın temel paketlerinde bulunur, ek paket gerektirmez
- Küçük ve orta ölçekli uygulamalar için yeterlidir
- Widget ağacında veri aktarımı için Provider gibi çözümler gerektirmez

Tema Değiştirme Uygulaması

Bu projede, kullanıcının aydınlık ve karanlık tema arasında geçiş yapabildiği basit bir uygulama geliştirilmiştir. Uygulama, AppBar'da bulunan bir Switch widget'ı ile tema değiştirme imkanı sunmaktadır.



Uygulamanın Özellikleri

- Aydınlık/Karanlık tema geçişi
- Tema durumunun uygulama genelinde saklanması
- Anlık tema değişimi
- Mevcut tema durumunun gösterilmesi

Kod Açıklaması

Aşağıda, tema değiştirme uygulamasının ana bileşenlerinin açıklaması yer almaktadır:

1. main.dart

```
1  import 'package:flutter/material.dart';
2
3  void main() {
4    runApp(MyApp());
5  }
6
7  class MyApp extends StatelessWidget {
8    // Tema modu için ValueNotifier
9    static final ValueNotifier<ThemeMode> themeNotifier =
10      ValueNotifier(ThemeMode.light);
11
12    @override
13    Widget build(BuildContext context) {
14      return ValueListenableBuilder<ThemeMode>(
15        valueListenable: themeNotifier,
16        builder: (_, ThemeMode currentMode, __) {
17          return MaterialApp(
18            title: 'Tema Değiştirici',
19            theme: ThemeData.light(),
20            darkTheme: ThemeData.dark(),
21            themeMode: currentMode,
22            home: HomePage(),
23          ); // MaterialApp
24        },
25      ); // ValueListenableBuilder
26    }
27  }
```

```

29 class HomePage extends StatelessWidget {
30   @override
31   Widget build(BuildContext context) {
32     bool isDark = MyApp.themeNotifier.value == ThemeMode.dark;
33
34     return Scaffold(
35       appBar: AppBar(
36         title: Text('Tema Toggle'),
37         actions: [
38           Switch(
39             value: isDark,
40             onChanged: (value) {
41               MyApp.themeNotifier.value =
42                 value ? ThemeMode.dark : ThemeMode.light;
43             },
44           ), // Switch
45         ],
46       ), // AppBar
47       body: Center(
48         child: Text(
49           'Şu anki tema: ${isDark ? 'Karanlık' : 'Aydınlık'}',
50           style: TextStyle(fontSize: 20),
51         ), // Text
52       ), // Center
53     ); // Scaffold
54   }
55 }

```

2. Kod Analizi

MyApp Sınıfı

- ValueNotifier<ThemeMode> statik değişkeni, uygulamanın tema durumunu saklar
- Başlangıçta aydınlık tema (ThemeMode.light) ayarlanmıştır
- ValueListenableBuilder widget'ı, tema değişikliklerini dinler
- MaterialApp, hem aydınlık hem karanlık tema tanımlarını içerir
- themeMode parametresi, aktif temayı belirler

HomePage Sınıfı

- isDark değişkeni, mevcut tema durumunu kontrol eder
- AppBar'daki Switch widget'ı, tema geçişi için kullanılır
- onChanged callback fonksiyonu, tema değiştirme mantığını içerir
- Ekranın ortasındaki Text widget'ı, mevcut tema durumunu gösterir

3. Durum Yönetimi Akışı

1. Uygulama başlatıldığında, ValueNotifier aydınlık tema değeriyle oluşturulur
2. Switch'in değeri, mevcut tema durumuna göre ayarlanır
3. Kullanıcı Switch'i değiştirdiğinde, ValueNotifier'ın değeri güncellenir
4. ValueListenableBuilder, değişikliği algılar ve MaterialApp'i yeniden oluşturur
5. themeMode güncellenir ve tüm uygulama yeni temayı uygular

Diğer Durum Yönetimi Alternatifleri

Bu projede ValueNotifier kullanılmıştır, ancak Flutter'da kullanılabilecek diğer popüler durum yönetimi yaklaşımları da bulunmaktadır:

Provider

Provider, Flutter ekibi tarafından önerilen, InheritedWidget üzerine inşa edilmiş bir durum yönetimi çözümüdür.

Özellikleri:

- Bağımlılık enjeksiyonu sağlar
- Widget ağacında veri paylaşımını kolaylaştırır
- ChangeNotifier sınıfı ile reaktif programlamayı destekler
- Farklı provider türleri sunar (ChangeNotifierProvider, StreamProvider, FutureProvider vb.)

Riverpod

Provider'ın yaratıcısı tarafından geliştirilen, Provider'ın bazı sınırlamalarını aşmak için tasarlanmış gelişmiş bir durum yönetimi paketidir.

Özellikleri:

- Compile-time güvenliği sağlar
- Birden fazla provider örneğine izin verir

- Provider referanslarını doğrudan kullanabilme
- Auto-dispose ve override özellikleri

Bloc/Cubit

Reactive Programming ve Clean Architecture prensiplerine dayanan, iş mantığını UI'dan tamamen ayıran yapısal bir durum yönetimi yaklaşımıdır.

Özellikleri:

- Event-State mimarisi
- İş mantığını UI'dan tamamen ayırma
- Öngörülebilir tek yönlü veri akışı
- Test edilebilirliği kolaylaştırır
- Büyük uygulamalar için ölçeklenebilir

GetX

Durum yönetimi, route yönetimi ve bağımlılık enjeksiyonu gibi birçok özelliği içeren kapsamlı bir Flutter paketidir.

Özellikleri:

- Minimum kod ile hızlı geliştirme
- Performans odaklı yaklaşım
- Reaktif ve basit sözdizimi
- Entegre route yönetimi ve bağımlılık enjeksiyonu

Redux

Merkezi bir store kullanarak tek yönlü veri akışı sağlayan, tahmin edilebilir durum güncellemeleri sunan bir mimaridir.

Özellikleri:

- Merkezi durum deposu
- Saf fonksiyonlarla durum güncellemeleri (reducers)
- Öngörülebilir tek yönlü veri akışı
- Kapsamlı izleme ve debugging

MobX

Observable programlama prensiplerine dayanan reaktif durum yönetimi yaklaşımıdır.

Özellikleri:

- Observer/Observable deseni
- Otomatik bağımlılık izleme
- Reaktif programlama
- Code generation ile kolay kullanım

Neden ValueNotifier Tercih Edildi?

Bu projede ValueNotifier'ın tercih edilmesinin birkaç önemli nedeni vardır:

1. **Basitlik:** ValueNotifier, Flutter'ın temel paketinde yer alan, ek bağımlılık gerektirmeyen basit bir çözümdür. Temaya geçiş gibi tek bir değişkeni izleme durumları için idealdir.
2. **Düşük Öğrenme Eğrisi:** Provider, Bloc gibi diğer çözümlere kıyasla öğrenmesi ve uygulaması daha kolaydır.
3. **Minimal Kod:** Tema değiştirme gibi basit durumlar için, daha karmaşık durum yönetimi paketlerinin tüm yapısını kurmaya gerek kalmadan çözüm sağlar.
4. **Performans:** Basit bir değer değişikliğini izlemek için yeterli performansı sağlar, gereksiz yeniden yapılanmaları önler.
5. **Entegrasyon Kolaylığı:** Mevcut Flutter projelere kolayca entegre edilebilir, büyük değişiklikler gerektirmez.
6. **Bağımsızlık:** Herhangi bir üçüncü parti paket gerektirmediği için, paket bağımlılıklarından ve uyumluluk sorunlarından kaçınılmış olur.

Bununla birlikte, uygulama büyüdükçe ve daha karmaşık durum yönetimi gerektikçe, yukarıda belirtilen diğer çözümlere geçiş yapmak daha uygun olabilir. Özellikle çok sayıda durum değişkeni ve karmaşık iş mantığı içeren uygulamalar için Provider, Riverpod veya Bloc gibi daha yapısal çözümler tercih edilebilir.

Sonuç

Flutter'da durum yönetimi, uygulamaların davranışını ve görünümünü kontrol etmek için kritik öneme sahiptir. Bu projede, ValueNotifier kullanarak basit ve etkili bir tema değiştirme mekanizması uygulanmıştır.

Bu yaklaşımın temel avantajları:

- Minimal kod ile temiz bir çözüm sunar
- Ek paket gerektirmez
- Küçük ve orta ölçekli uygulamalar için idealdir
- Widget ağacında kolay veri paylaşımı sağlar

Bununla birlikte, uygulama büyüdükçe ve karmaşıktıkça Provider, Riverpod, Bloc gibi daha ölçeklenebilir durum yönetimi çözümlerinin değerlendirilmesi önerilir.

Bu projede gösterilen tema değiştirme uygulaması, Flutter'da durum yönetiminin temel prensiplerini anlamak için iyi bir başlangıç noktasıdır. Durum yönetimi, Flutter uygulamalarının temel unsurlarından biridir ve farklı yaklaşımların anlaşılması, daha etkili ve bakımı kolay uygulamalar geliştirmek için önemlidir.

Kaynaklar

1. Flutter Resmi Dokümantasyonu - flutter.dev
2. Flutter State Management - flutter.dev/docs/development/data-and-backend/state-mgmt/intro
3. ValueNotifier API - api.flutter.dev/flutter/foundation/ValueNotifier-class.html
4. Flutter Theme - api.flutter.dev/flutter/material/ThemeData-class.html